

COURSE NAME: THEORY OF COMPUTATION

COURSE CODE: CSA1304

CO3 – AT2 : Lab Experiments

1. $S \rightarrow 0A1$, $A \rightarrow 0A \mid 1A \mid \epsilon$

Write a C program to check whether a given string belongs to the language defined by this CFG.

Language generated

A generates any string of 0's and 1's (including ϵ), so $S \rightarrow 0A1$ generates exactly the strings that start with 0 and end with 1:

$$L(G) = 0(0+1)^*1 = \{ \text{strings over } \{0,1\} \text{ that begin with 0 and end with 1} \}$$

Approach

Define $\text{matchA}(i,j)$ / $\text{matchS}(i,j)$ = "can the non-terminal derive the substring $s[i..j-1]$ ", built directly from the productions (this mirrors top-down CFG parsing, one production application per recursive call):

- $\text{matchS}(i,j)$: true iff $j-i \geq 2$, $s[i]='0'$, $s[j-1]='1'$, and $\text{matchA}(i+1, j-1)$ — this is exactly production $S \rightarrow 0A1$.
- $\text{matchA}(i,j)$: true if $i=j$ (ϵ), or $s[i] \in \{0,1\}$ and $\text{matchA}(i+1,j)$ — exactly productions $A \rightarrow 0A \mid 1A \mid \epsilon$.

C Program (compiled and tested with gcc)

```
#include <stdio.h>
#include <string.h>

char s[1000];

/* Does A derive the substring s[i..j-1] ? (A -> 0A | 1A | eps) */
int matchA(int i, int j) {
    if (i == j) return 1;                /* A -> eps */
    if (s[i] == '0' || s[i] == '1')     /* A -> 0A or A -> 1A */
        return matchA(i + 1, j);
    return 0;
}

/* Does S derive the substring s[i..j-1] ? (S -> 0A1) */
int matchS(int i, int j) {
    if (j - i < 2) return 0;             /* need at least "01" */
```

```

    if (s[i] != '0' || s[j - 1] != '1') return 0; /* first 0, last 1 */
    return matchA(i + 1, j - 1); /* middle must be A */
}

int main(void) {
    printf("Enter a string over {0,1}: ");
    scanf("%s", s);
    int n = strlen(s);

    for (int i = 0; i < n; i++) {
        if (s[i] != '0' && s[i] != '1') {
            printf("Invalid symbol in input (alphabet is {0,1}).\n");
            return 0;
        }
    }

    if (matchS(0, n))
        printf("\"%s\" belongs to the language L(G).\n", s);
    else
        printf("\"%s\" does NOT belong to the language L(G).\n", s);
    return 0;
}

```

Sample runs

Input string	Program output
01	"01" belongs to the language L(G).
0011	"0011" belongs to the language L(G).
0101011	"0101011" belongs to the language L(G).
0	"0" does NOT belong to the language L(G).
10	"10" does NOT belong to the language L(G).

2. $S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$

Write a C program to check whether a given string belongs to the language defined by this CFG.

Language generated

Each application of $S \rightarrow 0S0$ or $S \rightarrow 1S1$ wraps the same symbol on both ends, and generation stops with $S \rightarrow 0$, $S \rightarrow 1$, or $S \rightarrow \epsilon$ (an odd-length centre or an even-length empty centre). This produces exactly the palindromes over $\{0,1\}$:

$L(G) = \{ w \in \{0,1\}^* \mid w = \text{reverse}(w) \}$ (all binary palindromes, including ϵ)

Approach

`matchS(i,j)` checks the two boundary symbols and recurses inward — this directly mirrors $S \rightarrow 0S0 / 1S1$ peeling one matching pair of symbols per call, stopping at $S \rightarrow \epsilon$ or $S \rightarrow 0/1$:

- $i = j \rightarrow \text{true}$ ($S \rightarrow \epsilon$).
- $j - i = 1 \rightarrow \text{true}$ ($S \rightarrow 0$ or $S \rightarrow 1$, any single symbol is a 1-letter palindrome).
- otherwise: true iff $s[i] = s[j-1]$ and `matchS(i+1, j-1)` — the outer pair matches production $0S0$ or $1S1$.

C Program (compiled and tested with gcc)

```
#include <stdio.h>
#include <string.h>

char s[1000];

/* Does S derive the substring s[i..j-1] ? */
int matchS(int i, int j) {
    if (i == j) return 1;          /* S -> eps          */
    if (j - i == 1) return 1;      /* S -> 0 or S -> 1  */
    if (s[i] != s[j - 1]) return 0; /* outer symbols must match */
    return matchS(i + 1, j - 1);   /* S -> 0S0 or S -> 1S1 */
}

int main(void) {
    printf("Enter a string over {0,1}: ");
    scanf("%s", s);
    int n = strlen(s);

    for (int i = 0; i < n; i++) {
        if (s[i] != '0' && s[i] != '1') {
            printf("Invalid symbol in input (alphabet is {0,1}).\n");
        }
    }
}
```

```

        return 0;
    }
}

if (matchS(0, n))
    printf("\"%s\" belongs to the language L(G).\n", s);
else
    printf("\"%s\" does NOT belong to the language L(G).\n", s);
return 0;
}

```

Sample runs

Input string	Program output
0110	"0110" belongs to the language L(G).
101	"101" belongs to the language L(G).
0100010	"0100010" belongs to the language L(G).
	"" belongs to the language L(G).
100	"100" does NOT belong to the language L(G).

3. $S \rightarrow 0S0 \mid A, A \rightarrow 1A \mid \epsilon$

Write a C program to check whether a given string belongs to the language defined by this CFG.

Language generated

$S \rightarrow 0S0$ peels one matching 0 off each end, any number of times, and then switches (once) to A , which generates any number of 1's. This forces the same count of 0's on the left and right, with only 1's allowed in between:

$$L(G) = \{ 0^n 1^m 0^n \mid n \geq 0, m \geq 0 \}$$

(Not regular — recognising it needs to count and match the leading/trailing 0's, which is exactly why the C solution uses recursion instead of a simple linear scan.)

Approach

- $\text{matchS}(i,j)$: first try production $S \rightarrow 0S0$ — if $s[i]='0'$, $s[j-1]='0'$ and $\text{matchS}(i+1,j-1)$ succeeds, accept.
- Otherwise fall through to production $S \rightarrow A$, i.e. try $\text{matchA}(i,j)$.
- $\text{matchA}(i,j)$: true if $i=j$ (ϵ), or $s[i]='1'$ and $\text{matchA}(i+1,j)$ — productions $A \rightarrow 1A \mid \epsilon$.

C Program (compiled and tested with gcc)

```
#include <stdio.h>
#include <string.h>

char s[1000];

/* Does A derive s[i..j-1] ? (A -> 1A | eps) */
int matchA(int i, int j) {
    if (i == j) return 1;          /* A -> eps */
    if (s[i] == '1') return matchA(i + 1, j); /* A -> 1A */
    return 0;
}

/* Does S derive s[i..j-1] ? (S -> 0S0 | A) */
int matchS(int i, int j) {
    if (j - i >= 2 && s[i] == '0' && s[j - 1] == '0')
        if (matchS(i + 1, j - 1)) /* S -> 0S0 */
            return 1;
    return matchA(i, j);          /* S -> A */
}

int main(void) {
```

```

printf("Enter a string over {0,1}: ");
scanf("%s", s);
int n = strlen(s);

for (int i = 0; i < n; i++) {
    if (s[i] != '0' && s[i] != '1') {
        printf("Invalid symbol in input (alphabet is {0,1}).\n");
        return 0;
    }
}

if (matchS(0, n))
    printf("\"%s\" belongs to the language L(G).\n", s);
else
    printf("\"%s\" does NOT belong to the language L(G).\n", s);
return 0;
}

```

Sample runs

Input string	Program output
0110	"0110" belongs to the language L(G).
00	"00" belongs to the language L(G).
010	"010" belongs to the language L(G).
0011000	"0011000" does NOT belong to the language L(G). (2 leading 0s $\neq$ 3 trailing 0s)
0100	"0100" does NOT belong to the language L(G). (1 leading 0 $\neq$ 2 trailing 0s)

4. $S \rightarrow 0S1 \mid \epsilon$

Write a C program to check whether a given string belongs to the language defined by this CFG.

Language generated

Each application of $S \rightarrow 0S1$ adds one 0 at the front and one 1 at the back, terminating with $S \rightarrow \epsilon$ — the classic non-regular context-free language:

$$L(G) = \{ 0^n 1^n \mid n \geq 0 \}$$

Approach

`matchS(i,j)` peels a matching (0 ... 1) pair from the two ends per call, exactly mirroring $S \rightarrow 0S1$, and bottoms out at $S \rightarrow \epsilon$:

- $i = j \rightarrow \text{true}$ ($S \rightarrow \epsilon$).
- otherwise: true iff $j - i \geq 2$, $s[i] = '0'$, $s[j-1] = '1'$, and `matchS(i+1, j-1)`.

C Program (compiled and tested with gcc)

```
#include <stdio.h>
#include <string.h>

char s[1000];

/* Does S derive s[i..j-1] ? */
int matchS(int i, int j) {
    if (i == j) return 1;                /* S -> eps */
    if (j - i < 2) return 0;
    if (s[i] != '0' || s[j - 1] != '1') return 0;
    return matchS(i + 1, j - 1);         /* S -> 0S1 */
}

int main(void) {
    printf("Enter a string over {0,1}: ");
    scanf("%s", s);
    int n = strlen(s);

    for (int i = 0; i < n; i++) {
        if (s[i] != '0' && s[i] != '1') {
            printf("Invalid symbol in input (alphabet is {0,1}).\n");
            return 0;
        }
    }
}
```

```

if (matchS(0, n))
    printf("\n%s\ belongs to the language L(G).\n", s);
else
    printf("\n%s\ does NOT belong to the language L(G).\n", s);
return 0;
}

```

Sample runs

Input string	Program output
0011	"0011" belongs to the language L(G).
01	"01" belongs to the language L(G).
000111	"000111" belongs to the language L(G).
	"" belongs to the language L(G).
0110	"0110" does NOT belong to the language L(G).
0001111	"0001111" does NOT belong to the language L(G). (unequal counts)

5. $S \rightarrow A101A$, $A \rightarrow 0A \mid 1A \mid \varepsilon$

Write a C program to check whether a given string belongs to the language defined by this CFG.

Language generated

A generates any binary string (including ε), so $S \rightarrow A101A$ generates any string that contains the literal substring "101" somewhere, with arbitrary binary strings on either side:

$$L(G) = (0+1)^* 101 (0+1)^* = \{ w \in \{0,1\}^* \mid "101" \text{ occurs as a substring of } w \}$$

Approach

- `matchA(i,j)` is trivially true for any substring made only of 0's and 1's, since $A \rightarrow 0A \mid 1A \mid \varepsilon$ can generate any binary string.
- `matchS(i,j)` tries every position `p` in the substring where the fixed middle part "101" (from production $S \rightarrow A101A$) could sit, and accepts if some such `p` works together with `matchA` on both sides.
- Since `matchA` is always true here, this effectively (and correctly) reduces to: "does the substring contain 101 anywhere?" — directly reflecting what the grammar generates.

C Program (compiled and tested with gcc)

```
#include <stdio.h>
#include <string.h>

char s[1000];

/* A -> 0A | 1A | eps generates ANY string over {0,1}, so A always
   derives s[i..j-1] as long as every character in it is 0 or 1. */
int matchA(int i, int j) {
    for (int k = i; k < j; k++)
        if (s[k] != '0' && s[k] != '1') return 0;
    return 1;
}

/* S -> A 1 0 1 A : try every position p where the literal "101" could sit */
int matchS(int i, int j) {
    for (int p = i; p + 3 <= j; p++) {
        if (s[p] == '1' && s[p + 1] == '0' && s[p + 2] == '1' &&
            matchA(i, p) && matchA(p + 3, j))
            return 1;
    }
    return 0;
}
```

```

}

int main(void) {
    printf("Enter a string over {0,1}: ");
    scanf("%s", s);
    int n = strlen(s);

    for (int i = 0; i < n; i++) {
        if (s[i] != '0' && s[i] != '1') {
            printf("Invalid symbol in input (alphabet is {0,1}).\n");
            return 0;
        }
    }

    if (matchS(0, n))
        printf("\"%s\" belongs to the language L(G).\n", s);
    else
        printf("\"%s\" does NOT belong to the language L(G).\n", s);
    return 0;
}

```

Sample runs

Input string	Program output
101	"101" belongs to the language L(G).
0101	"0101" belongs to the language L(G).
00101011	"00101011" belongs to the language L(G).
111000	"111000" does NOT belong to the language L(G).
110	"110" does NOT belong to the language L(G).
10	"10" does NOT belong to the language L(G).